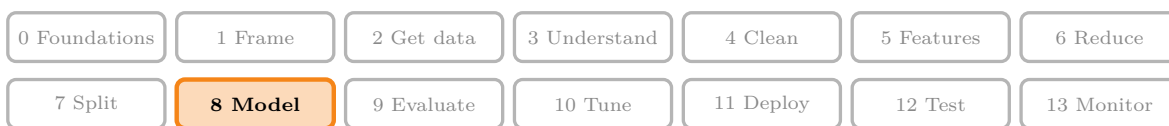


What a Convolutional Neural Network Is

Contents

1. Overview	2
2. Prerequisites	3
3. What makes a network a CNN	3
4. Why not just use an ANN on images?	3
4.1 An image is a grid of numbers	3
4.2 Problem 1: too many weights	4
4.3 Problem 2: overfitting	5
4.4 Problem 3: the spatial arrangement is lost	5
5. How a CNN recognises an image	6
5.1 Breaking a 9 into features	6
5.2 Layer by layer	7
6. Where CNNs are used	8
7. Summary	9
8. Sources	9
9. Key terms	9

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Representation learning ([Note 1002](#)).
- **Compare with:** Multi-layer perceptron (MLP) ([Note 1009](#)).

1. Overview

Key point: A convolutional neural network (CNN) is a neural network built for grid-like data such as images. Instead of connecting every pixel to every node, it slides small filters over the image to find simple features like edges, then combines them, layer by layer, into more complex features until it can recognise the whole object.

A **convolutional neural network** (CNN, or ConvNet) is “a specialized kind of neural network for processing data that has a known grid-like topology” (Goodfellow et al. 2016, ch. 9). Time series are a 1D grid: one value after another at regular time steps. Images are a 2D grid of pixels. Whenever data has such a grid structure, a CNN can be applied, and it usually works very well.

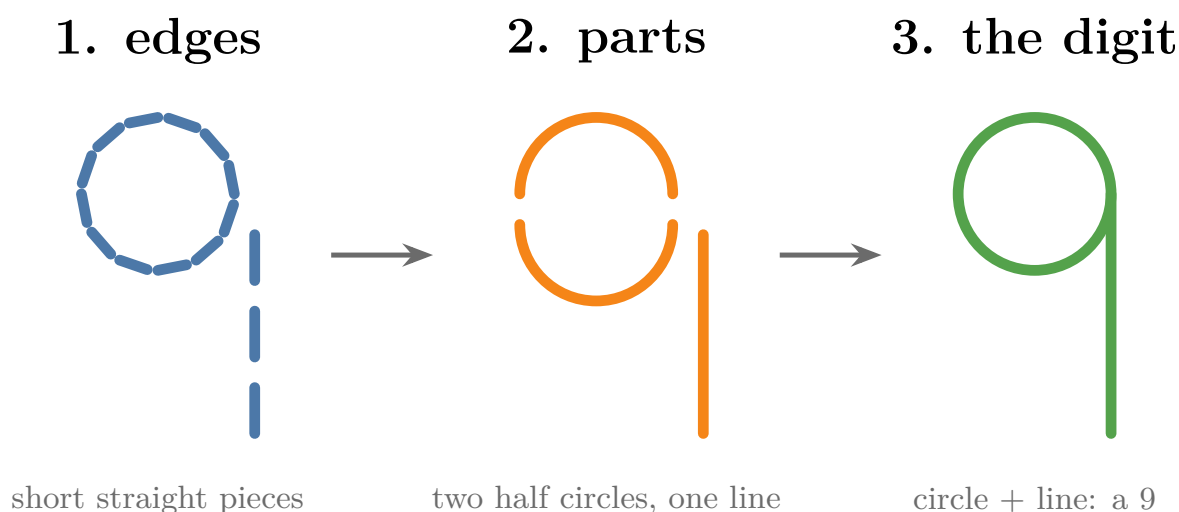


Figure 1: How a CNN sees a 9: first short edges, then parts made of edges, then the digit made of parts

This Note covers what makes a network a CNN, why a plain ANN struggles with images, how a CNN builds features step by step (Figure 1), and where CNNs are used.

2. Prerequisites

- The [MLP intuition Note](#): layers of nodes, each combining the outputs of the previous layer.
- The [MNIST ANN Note](#): handwritten digits classified by an ANN with a Flatten layer.
- The [overfitting Note](#): a model that fits the training data too closely and fails on new data.

3. What makes a network a CNN

Key point: A CNN contains at least one convolution layer. A CNN is usually built from three kinds of layers: convolution, pooling and fully connected.

A CNN is a neural network whose architecture differs slightly from an ANN's. The difference is a special layer, the **convolution layer**, which performs a special operation, the **convolution operation**. An ANN combines its inputs by matrix multiplication; a CNN uses convolution, in at least one layer (Goodfellow et al. 2016, ch. 9). So if a network's architecture contains a convolution layer, it is a CNN.

A CNN is built from three kinds of layers:

1. **Convolution layers:** they slide small filters over the image to find features. Taught in the [convolution operation Note](#).
2. **Pooling layers:** they shrink the result. Taught in the [pooling Note](#).
3. **Fully connected layers:** ordinary Dense layers, in which every node is connected to every node of the next layer, as in the [MNIST ANN Note](#). They are often called **FC layers**.

The design of CNNs was inspired by the visual cortex, the part of the brain we see with (see the [CNN and visual cortex Note](#)).

4. Why not just use an ANN on images?

Key point: An ANN can classify images, but flattening an image costs a huge number of weights, invites overfitting, and throws away where each pixel is.

An ANN can work on images: the [MNIST ANN Note](#) reached 97.63% test accuracy on handwritten digits. But a CNN usually does better on images, because an ANN has three problems with them.

4.1 An image is a grid of numbers

Key point: A greyscale image is a 2D grid; each cell (pixel) holds one number from 0 (black) to 255 (white).

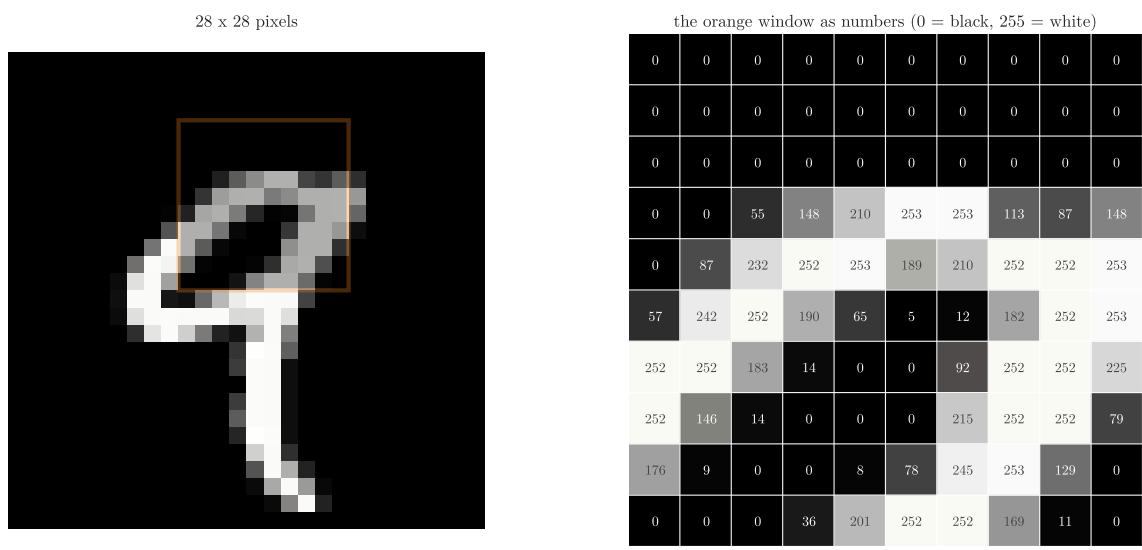


Figure 2: An MNIST 9 (left) and a 10×10 window of it (right) as the numbers the computer stores: 0 is black, 255 is white

To a computer, an image is a 2D grid of **pixels**, and each pixel holds a number for its brightness (Figure 2). An MNIST digit is $28 \times 28 = 784$ pixels. Different numbers in different pixels are what make us see a shape.

To feed such an image to an ANN, we flatten it: the first row of pixels, then the second row after it, and so on, into one long row of 784 inputs (the Flatten layer of the [MNIST ANN Note](#)). Every input is then connected to every node of the first hidden layer.

4.2 Problem 1: too many weights

Key point: The first Dense layer has (number of pixels) $\times$ (number of nodes) weights. A 40×40 image and 100 nodes already need 160,000; a 1000×1000 image and 500 nodes need 500 million.

1. **In words:** every pixel is connected to every node, so the weights multiply.
2. **Formula:**

$$\text{weights of the first layer} = \text{height} \times \text{width} \times \text{nodes}$$

3. **Example:** a 40×40 image flattened gives 1,600 inputs. With a small hidden layer of 100 nodes: $1,600 \times 100 = 160,000$ weights, for a tiny image and a tiny layer. A 1000×1000 image with 500 nodes: $1,000,000 \times 500 = 500,000,000$ weights.

Every one of these weights must be stored, used in forward propagation and updated by backpropagation. As images grow, the weights grow, and training becomes very slow and costly on a large dataset.

4.3 Problem 2: overfitting

Key point: With so many connections, the network can memorise tiny details of the training images instead of learning patterns that hold on new ones.

Connecting every pixel to every node gives the network an enormous number of weights to fit, so it tries to capture every minute pattern of the training images. The network then does well on the training data but worse on test data: it **overfits** (see the [overfitting Note](#)). The CS231n notes make the same point: full connectivity on images “is wasteful and the huge number of parameters would quickly lead to overfitting”.

4.4 Problem 3: the spatial arrangement is lost

Key point: Flattening separates neighbouring pixels; the ANN never learns which pixels were next to which. In the Notebook an ANN learns scrambled images, unreadable to us, just as well as normal ones.

In a 2D image, where each pixel sits carries meaning. In a photo of a monkey, the eyes are above the nose, and the distance between them is an important clue that this is a monkey’s face. Flattening row by row destroys this arrangement: a pixel and the one just below it end up 28 places apart in the flattened row (Figure 3). The concept of distance in 2D no longer exists in 1D.

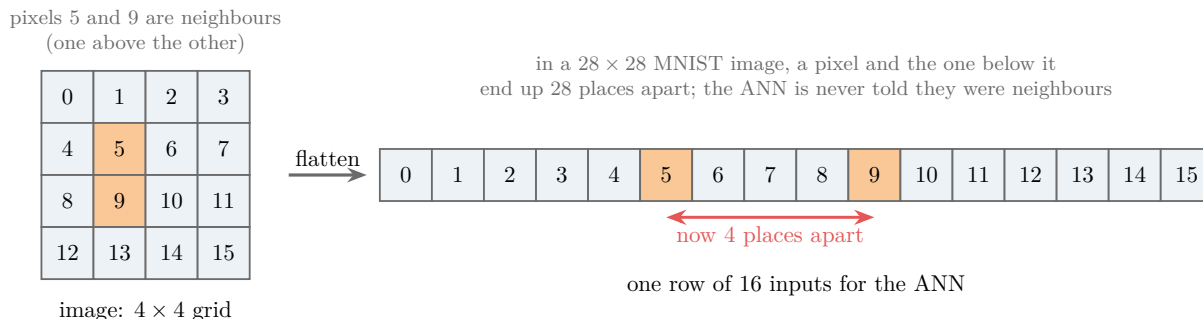


Figure 3: Flattening a 4×4 grid: pixels 5 and 9, one above the other, end up 4 places apart. In a 28×28 image they end up 28 places apart

The Notebook shows that an ANN really does not use the arrangement. We take one fixed random shuffle of the 784 pixel positions and apply it to every training and test image. The scrambled digits are unreadable to us (Figure 4), yet the same ANN, trained the same way (5 epochs, 3 seeds), reaches 97.39% test accuracy on them, against 97.36% on the normal images.

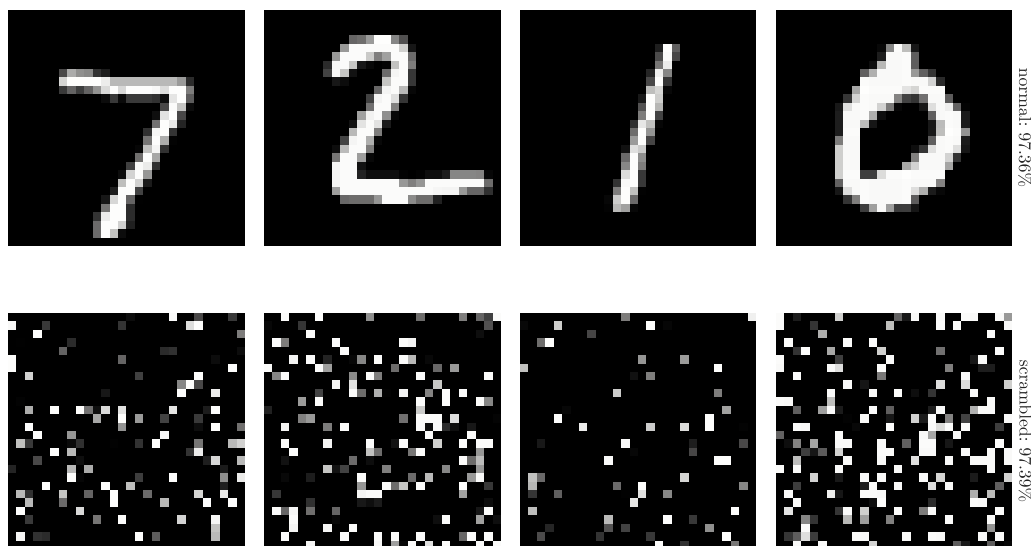


Figure 4: Top: four test digits. Bottom: the same digits with their pixels scrambled by one fixed permutation. The ANN’s mean test accuracy is the same on both

An ANN treats its 784 inputs as an unordered list: shuffle them consistently and it learns the same. It cannot use the fact that some pixels are neighbours, which is exactly the information that makes an image an image. A CNN’s filters look at small patches of neighbouring pixels, so they use this structure directly.

5. How a CNN recognises an image

Key point: Early layers find primitive features such as edges; each later layer combines the previous layer’s features into more complex ones, until the last layers can recognise the whole object.

5.1 Breaking a 9 into features

Key point: We recognise a 9 as a circle on top of a vertical line, whatever the handwriting. A CNN works the same way.

Suppose we must say whether an image shows a 9. The task is not simple: everyone writes a 9 differently, and the model must recognise all of them.

How do we do it ourselves? We look for patterns: a circle at the top and a vertical line down the right side (Figure 1). Even if the circle is a little squashed or the line a little slanted, we still see a 9. We break the digit into features and check that the right features are present.

A CNN follows the same principle. Given the image, it first extracts **primitive features: edges**, the short straight pieces that make up every stroke, such as the many small edges

that form the circle. Then, layer by layer, it combines them into more complex features: two half circles, then a full circle with a line below it, and finally the 9.

5.2 Layer by layer

Key point: Filters in the first convolution layer detect edges; the next convolution layers merge them into larger, more meaningful features.

A convolution layer contains **filters**: small grids of numbers that extract features from the image by simple mathematical operations (see the [convolution operation Note](#)). Each filter moves over the image and checks, at every place, whether its pattern is there. Where the pattern is present, the filter's output is activated.

These activated features are passed to another convolution layer, whose filters merge them into more complex but more meaningful features. The deeper we go in the network, the more complex the features become, until the last layers hold the features that decide whether the digit is a 9.

Figure 5 shows this in a small CNN trained in the Notebook (two convolution layers, 97.72% test accuracy after 3 epochs). Watch the maps grow coarser and more specific from layer to layer. Layer 1's filters see 3×3 pixels; a filter with a red side and a blue side, such as the first, lights up along edges of one direction. Layer 2 works on 2×2 -pooled layer-1 maps, so each of its 3×3 filters covers 8×8 pixels of the digit, and its maps mark larger pieces: the stem, the loop, the top stroke.

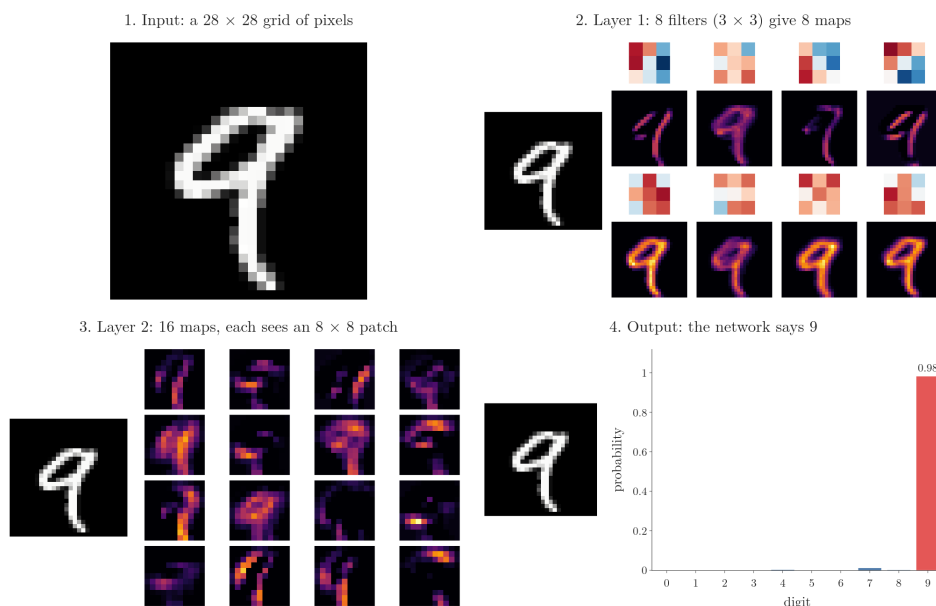


Figure 5: A 9 through a small trained CNN. (1) The input. (2) Layer 1: each 3×3 filter (red positive, blue negative weights) and the map it gives. (3) Layer 2: 16 coarser maps that mark parts of the digit. (4) The output probabilities: 0.98 for 9

The same holds for a photo of a cat. The first layers detect edges. The next layers detect parts such as ears, eyes or a mouth. Later layers combine eyes and ears into a face, and the face and body into a cat.

Extra: A real CNN trained on MNIST does learn edge detectors in its first layer without being told to: the [convolution operation Note](#), section 7, trains one and finds vertical- and horizontal-edge filters among its learned filters.

6. Where CNNs are used

Key point: Image classification, object localisation and detection, face recognition, image segmentation, super-resolution, colourisation and pose estimation.

CNNs are among the most successful neural networks in real-world use, from face recognition software to self-driving cars. Some application areas:

Task	What the CNN does
Image classification	Says which class an image belongs to, such as cat or dog
Object localisation	Finds where an object is and draws a rectangular box around it
Object detection	Finds and boxes every object in an image, with a confidence for each; used in self-driving cars
Face detection and recognition	Finds faces and recognises whose they are, as in smartphone cameras
Image segmentation	Divides an image into regions, such as a tiger and the grass behind it
Super-resolution	Raises the resolution of low-quality images, such as old photos
Colourisation	Turns black-and-white photos and films into colour
Pose estimation	Detects the posture of a person's body from a camera feed, as in fitness apps and motion-controlled games

7. Summary

	ANN on images	CNN
Input	flattened to 1D	kept as a 2D grid
Weights	pixels $\times$ nodes; huge for big images	small filters
Overfitting	likely, with so many weights	less likely
Spatial arrangement	lost (scrambling the pixels changes nothing)	used: filters look at neighbouring pixels
Features	none built in	edges, then parts, then objects

- A CNN is a neural network for grid-like data, with at least one convolution layer.
- It is built from convolution layers, pooling layers and fully connected layers.
- An ANN on images needs huge numbers of weights, overfits, and ignores where pixels are.
- A CNN finds edges first and combines them into more complex features, layer by layer.
- Its design was inspired by the visual cortex.

8. Sources

Built from

- CampusX, “What is Convolutional Neural Network (CNN) | CNN Intuition”, YouTube, <https://www.youtube.com/watch?v=hDVFXf74P-U>

Other references

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 9 introduction (definition of convolutional networks).
- Stanford CS231n course notes, “Convolutional Neural Networks”, section “Regular Neural Nets don’t scale well to full images”, cs231n.github.io/convolutional-networks.

9. Key terms

Term	Meaning
Convolutional neural network (CNN)	A neural network for grid-like data that uses convolution in at least one layer
Grid-like topology	Data arranged on a regular grid: 1D for time series, 2D for images
Pixel	One cell of an image grid, holding a brightness value
Convolution layer	A layer that slides small filters over its input to find features

Term	Meaning
Pooling layer	A layer that shrinks the output of a convolution layer
Fully connected (FC) layer	A Dense layer: every node connected to every node of the next layer
Primitive feature	A basic feature such as an edge, found by the first layers
Spatial arrangement	Where each pixel sits relative to the others